

Aim:

You are developing a real-time emergency ambulance routing system for Mumbai, one of India's busiest metropolitan cities.

- Road intersections are represented as graph vertices.
- Roads are represented as weighted edges, where the weight indicates travel time (affected by traffic density, road conditions, etc.).

When an emergency occurs, the system must instantly compute the shortest travel time from the hospital to all locations in the city to ensure ambulances reach patients as quickly as possible.

Your task is to implement Dijkstra's Algorithm to achieve this.

Input Format:

1. The first line contains two space-separated integers V and E , representing the number of vertices (road intersections) and edges (roads), respectively.
2. The next E lines each contain three space-separated integers u , v , and w , where:
 - $u \rightarrow$ Starting intersection
 - $v \rightarrow$ Destination intersection
 - $w \rightarrow$ Travel time between the two intersections
3. The last line contains an integer src , representing the source vertex (hospital location).

Output Format:

- Print V space-separated integers representing the shortest travel time from the source vertex to every vertex in the graph, with a trailing space after the last integer.

Note: Partial code is provided. Your task is to complete the Dijkstra's Algorithm logic and print the shortest distances from the source vertex to all other vertices.

Source Code:

ambulanceRouting.c

```
#include <stdio.h>
#include <limits.h>
#define MAX 100

int main() {
    int V, E;
    scanf("%d %d", &V, &E);
    int graph[MAX][MAX];

    for(int i = 0; i < V; i++)
        for(int j = 0; j < V; j++)
            graph[i][j] = INT_MAX;
    for(int i = 0; i < E; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        graph[u][v] = w;
        graph[v][u] = w;
    }
    int src;
```

```

// Type your logic here...
for(int i=0;i<V;i++){
    dist[i]=INT_MAX;
    visited[i]=0;
}
dist[src]=0;
for(int count=0;count<V-1;count++){
    int min_dist=INT_MAX;
    int u=-1;

    for(int v=0;v<V;v++){
        if(!visited[v] && dist[v] <= min_dist){
            min_dist=dist[v];
            u=v;
        }
    }

    if(u==-1){
        break;
    }
    visited[u]=1;

    for(int v=0;v<V;v++){
        if(!visited[v] && graph[u][v] != INT_MAX && dist[u]!=INT_MAX
&& dist[u]+graph[u][v]<dist[v]){
            dist[v]=dist[u]+graph[u][v];
        }
    }
}
for(int i=0;i<V;i++){
    if(dist[i]==INT_MAX){
        printf("-1");
    }else{
        printf("%d ",dist[i]);
    }
}
printf("\n");
return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5 6
0 1 2
0 2 4
1 2 1
1 3 7
2 4 3
3 4 1

0
0 2 3 7 6

Test Case - 2
User Output
3 3
0 1 5
1 2 2
0 2 10
0
0 5 7